

Lekcja

Temat: Asynchroniczność, synchroniczność w Pythonie

Synchroniczność oznacza wykonywanie instrukcji **jedna po drugiej** — program czeka aż dana operacja się zakończy, zanim przejdzie dalej.

```
import time

print("Start")
time.sleep(3)
print("Koniec")
```

Asynchroniczność pozwala wykonywać inne zadania w czasie oczekiwania na zakończenie operacji (np. pobierania danych z internetu, odczytu pliku, zapytania do bazy).

```
import asyncio

async def zadanie():
    print("Start")
    await asyncio.sleep(3)
    print("Koniec")

asyncio.run(zadanie())
```

Główne narzędzia:

- **Coroutines** — funkcje, które mogą być wstrzymywane i wznowiane (**async def**)
- **async/await** — składnia do pracy z coroutine'ami
- **asyncio** — pętla zdarzeń (**event loop**), która zarządza wszystkim

Przykład: **asyncio.sleep**

```
import asyncio
import time

def sync_sleep():
    print("Zaczynam spać sync...")
    time.sleep(2)
    print("Obudziłem się sync")

async def async_sleep():
    print("Zaczynam spać async...")
```

```
    await asyncio.sleep(2)
    print("Obudziłem się async")

async def main():
    await async_sleep()

asyncio.run(main())
```

Instalacja pip i aiohttp

1. Otwórz terminal / wiersz polecenia i wpisz:
python -m ensurepip --upgrade
python -m pip install aiohttp

Przykład

```
import asyncio
import aiohttp

async def main():
    url = "https://jsonplaceholder.typicode.com/users"

    async with aiohttp.ClientSession() as session:
        async with session.get(url) as response:
            print(f"Status odpowiedzi: {response.status}")

            users = await response.json()
            print(f"\nPobrano {len(users)} użytkowników\n")

            for user in users[:3]:
                print(f"• {user['name']}")
                print(f"  Email: {user['email']}")
                print(f"  Firma: {user['company']['name']}")
                print("-" * 40)

asyncio.run(main())
```

1. Coroutines (Korutyny) — async def

Coroutine to specjalna funkcja, którą można **wstrzymać** (await) i później **wznowić** w dowolnym momencie. Dzięki temu event loop może w tym czasie robić inne rzeczy.

```
import asyncio
```

```

import time

# Zwykła funkcja (synchronous)
def zwykla_funkcja():
    print("Zaczynam pracę")
    time.sleep(2)
    print("Skończyłem")

# Coroutine (asynchroniczna funkcja)
async def coroutine_przyklad():
    print("Zaczynam coroutine...")
    await asyncio.sleep(2)      # ← tu coroutine się wstrzymuje
    print("Coroutine wznowiona po 2 sekundach")
    return "Zakończono!"

# Uruchomienie
async def main():
    coro = coroutine_przyklad()    # ← tworzy coroutine (jeszcze nie działa!)
    print(type(coro))              # <class 'coroutine'>

    wynik = await coro             # ← teraz się wykonuje
    print(wynik)

asyncio.run(main())

```

Kluczowe cechy coroutines:

- Tworzy się przez `async def`
- Nie wykonuje się od razu po wywołaniu
- Musi być uruchomiona przez `await` lub `asyncio.run()`

2. **async/await** — składnia do pracy z coroutine'ami

`async` i `await` to **słowa kluczowe**, które razem tworzą asynchroniczny kod.

```

import asyncio

async def pobierz_dane():
    print("Zaczynam pobieranie...")
    await asyncio.sleep(1.5)      # await = wstrzymaj i oddaj kontrolę
    print("Dane pobrane!")
    return {"id": 1, "nazwa": "Jan"}

```

```
async def main():
    print("Start programu")

    # await = czekaj na wynik tej coroutiney
    dane = await pobierz_dane()

    print("Otrzymałem:", dane)
    print("Koniec programu")

asyncio.run(main())
```

Reguły:

- await można używać **tylko** wewnątrz async def
- await może czekać na inną coroutine, Task, lub obiekt z metodą `__await__()`

3. **asyncio** — pętla zdarzeń (Event Loop)

To jest **serce** całej asynchroniczności w Pythonie. Event Loop zarządza wszystkimi coroutine'ami, decyduje kiedy którą wznowić, obsługuje timer'y, I/O itp.

```
import asyncio
```

```
async def zadanie1():
    await asyncio.sleep(1)
    print("Zadanie 1 zakończone")
```

```
async def zadanie2():
    await asyncio.sleep(2)
    print("Zadanie 2 zakończone")
```

```
async def main():
    print("Event Loop zaczyna działać...")

    # Uruchamiamy wiele zadań "równolegle"
    await asyncio.gather(
        zadanie1(),
        zadanie2()
    )

    print("Event Loop zakończył wszystkie zadania")
```

```
# asyncio.run() → tworzy event loop, uruchamia main() i go zamyka
asyncio.run(main())
```

Porównanie wszystkich trzech razem

```
import asyncio

# 1. Coroutine
async def pobierz(url):
    print(f"[{url}] Rozpoczynam pobieranie...")
    await asyncio.sleep(2)          # symulacja opóźnienia sieci
    print(f"[{url}] Zakończono!")
    return f"dane z {url}"

# 2. async/await + 3. asyncio
async def main():
    # Tworzymy listę coroutine'ów
    coro1 = pobierz("https://api1.com")
    coro2 = pobierz("https://api2.com")
    coro3 = pobierz("https://api3.com")

    # Event Loop zarządza wykonaniem
    wyniki = await asyncio.gather(coro1, coro2, coro3)

    print("Wszystkie wyniki:", wyniki)

asyncio.run(main())
```

Lekcja

Temat: Uruchamianie wielu funkcji asynchronicznych w Pythonie. Event loop

Uruchamianie kilka tasków równolegle.

```
import asyncio

async def taskSleep4():
    print("taskSleep4 start")

    for i in range(3):
        print(f"taskSleep4 -> {i}")
        await asyncio.sleep(4)

    print("taskSleep4 koniec")

async def task2():
    print("Task 2 start")

    for i in range(3):
        print(f"Task 2 -> {i}")
        await asyncio.sleep(1)

    print("Task 2 koniec")

async def main():
    await asyncio.gather(
        taskSleep4(),
        task2()
    )

asyncio.run(main())
```

Event Loop w Pythonie to mechanizm, który zarządza zadaniami asynchronicznymi (async/await).

Można powiedzieć, że to:

„koordynator”, który pilnuje kiedy dane zadanie ma się wykonywać i kiedy program może przełączyć się na inne zadanie.

Najczęściej używa się go przez bibliotekę asyncio.

Event Loop:

1. uruchamia zadania (async def)
2. sprawdza czy jakieś zadanie czeka (await)
3. jeśli czeka → przełącza się na inne zadanie

4. wraca do zadania gdy operacja się zakończy

Przykład

```
import asyncio

async def zadanie1():
    print("Start zadanie 1")

    await asyncio.sleep(5)

    print("Koniec zadanie 1")

async def zadanie2():
    print("Start zadanie 2")

    await asyncio.sleep(1)

    print("Koniec zadanie 2")

async def main():
    await asyncio.gather(
        zadanie1(),
        zadanie2()
    )

asyncio.run(main())
```

KROK 1

Python widzi:

```
async def zadanie1():
```

i:

```
async def zadanie2():
```

To NIE uruchamia funkcji. Tworzy tylko coroutine functions.

„funkcje, które mogą być wykonywane przez event loop”

KROK 2

Dochodzi do:

```
asyncio.run(main())
```

Tutaj dzieje się kilka rzeczy:

```
asyncio.run():
```

1. tworzy **event loop**
2. uruchamia coroutine **main()**
3. zarządza całym asyncio
4. zamyka loop po zakończeniu

KROK 3

Event loop startuje:

```
main()
```

czyli:

```
async def main():
```

KROK 4

W main():

```
await asyncio.gather(  
    zadanie1(),  
    zadanie2()  
)
```

Co robi `gather()`?

`asyncio.gather()`:

1. tworzy task dla zadanie1
2. tworzy task dla zadanie2
3. uruchamia je współbieżnie
4. czeka aż oba się skończą

KROK 5

Event loop uruchamia zadanie1 Wykonuje:

```
print("Start zadanie 1")
```


wynik:

Start zadanie 1

KROK 6

Dochodzi do:

```
await asyncio.sleep(5)
```

I TO jest najważniejszy moment.

Coroutine mówi:

„pauzuję na 5 sekundy”

„event loop może robić coś innego”

zadanie1 zostaje zawieszone.

KROK 7

Event loop widzi:

„OK, zadanie1 śpi”

„sprawdzam inne taski”

uruchamia zadanie2

KROK 8

W zadanie2:

```
print("Start zadanie2")
```

wynik:

Start zadanie2

KROK 9

Dochodzi do:

```
await asyncio.sleep(1)
```

Coroutine mówi:

„śpię 1 sekundę”

„oddaję sterowanie”

zadanie2 zostaje zawieszona.

KROK 10

Teraz oba taski śpią:

Zadania	Czas
zadanie1	5 sekund
zadanie2	1 sekunda

Event loop czeka.

KROK 11

Po 1 sekundzie: budzi się zadanie2

Event loop wrzuca task z powrotem do kolejki.

KROK 12

Kontynuacja zadanie2

Wykonuje:

```
print("Koniec zadanie 2")
```

wynik:

Koniec zadanie2

zadanie2 kończy się.

KROK 13

Event loop dalej czeka na zadanie1, bo ono miało sleep 5 sekundy.

KROK 14

Po kolejnej sekundzie: budzi się zadanie1

kontynuuje:

```
print("Koniec zadanie 1")
```

wynik:

Koniec zadanie1

KROK 15

Oba taski zakończone.

gather() kończy działanie.

main() kończy działanie.

asyncio.run() zamyka event loop.

Program kończy się.

synchronicznie

```
import time
```

```
def zadanie1():  
    print("Start 1")  
    time.sleep(2)  
    print("Koniec 1")
```

```
def zadanie2():  
    print("Start 2")  
    time.sleep(1)  
    print("Koniec 2")
```

```
zadanie1()  
zadanie2()
```

Wersja SYNCHRONICZNA (requests)

Potrzebana instalacja

```
python -m pip install requests
```

```
import requests  
import time
```

```
def pobierz_dane_sync():  
    urls = [  
        "https://jsonplaceholder.typicode.com/users",
```

```

    "https://jsonplaceholder.typicode.com/posts",
    "https://jsonplaceholder.typicode.com/albums",
    "https://jsonplaceholder.typicode.com/todos",
]

wyniki = []
start = time.time()

for url in urls:
    print(f"Pobieram: {url}")
    response = requests.get(url)          # ← blokuje program
    dane = response.json()
    wyniki.append(len(dane))
    print(f"✓ Pobrano {len(dane)} rekordów")

czas = time.time() - start
print(f"\nSynchronicznie zakończono w: {czas:.2f} sekund")

```

pobierz_dane_sync()

Charakterystyka:

- Kod jest prostszy i łatwiejszy do zrozumienia
- Zapytania wykonują się **po kolei** (kolejno)
- Cały program czeka na każde zapytanie

Wersja ASYNCHRONICZNA (aiohttp)

```

import asyncio
import aiohttp
import time

async def pobierz(url: str, session: aiohttp.ClientSession):
    async with session.get(url) as response:
        dane = await response.json()
        print(f"✓ Pobrano {len(dane)} rekordów z {url.split('/')[-1]}")
        return len(dane)

async def main():
    urls = [
        "https://jsonplaceholder.typicode.com/users",
        "https://jsonplaceholder.typicode.com/posts",
        "https://jsonplaceholder.typicode.com/albums",
        "https://jsonplaceholder.typicode.com/todos",
    ]

```

```
]
```

```
start = time.time()
```

```
async with aiohttp.ClientSession() as session:
    # Uruchamiamy wszystkie zapytania równolegle
    zadania = [pobierz(url, session) for url in urls]
    wyniki = await asyncio.gather(*zadania)

czas = time.time() - start
print(f"\nAsynchronicznie zakończono w: {czas:.2f} sekund")
```

```
asyncio.run(main())
```

Lekcja

Temat: Wielowątkowość i wieloprocusowość: Threading, multiprocessing, GIL.

W Pythonie wielowątkowość różni się od innych języków ze względu na **Globalny Blok Interpretera (GIL)**. Oto kluczowe informacje:

GIL (Global Interpreter Lock)

- Mechanizm **uniemożliwiający równoczesne wykonywanie kodu Python przez wiele wątków**
- **W danym momencie tylko jeden wątek** wykonuje kod Python
- Chroni przed problemami z pamięcią, ale ogranicza rzeczywistą równoległość

threading – Wielowątkowość

```
import threading
import time
```

```
def io_task(name, duration):
    print(f"Zadanie {name} rozpoczęte")
    time.sleep(duration) # Symulacja I/O
    print(f"Zadanie {name} zakończone")
```

```
# Tworzenie wątków
thread1 = threading.Thread(target=io_task, args=("A", 2))
thread2 = threading.Thread(target=io_task, args=("B", 1))
...

    • args=("A", 2) - argumenty dla funkcji (krotka)
      o "A" → pierwszy argument name w funkcji
      o 2 → drugi argument duration w funkcji

...

thread1.start()
thread2.start()

# Czekanie na zakończenie
thread1.join()
thread2.join()
```

Wyjaśnienie:

```
Główny: startuję wątki
A: start
B: start
Główny: czekam na zakończenie
B: koniec (trwało 1s)
A: koniec (trwało 2s)
Główny: koniec programu
```

Wątki są zarządzane przez system operacyjny - to **nie Python** decyduje który wątek działa kiedy.
Możliwe scenariusze:

1. **W2 może skończyć przed W1** (jak wyżej)
2. **W1 może zacząć pierwszy** - różne czasy

Synchronizacja wątków

To sposób kontrolowania dostępu wielu wątków do wspólnych zasobów (np. zmiennych, plików, struktur danych), żeby uniknąć błędów typu „race condition” (wyścig wątków).

Problem bez synchronizacji

Jeśli dwa wątki jednocześnie modyfikują tę samą zmienną, wynik może być nieprzewidywalny.

```

import threading

counter = 0

def increment():
    global counter
    for _ in range(100000):
        counter += 1

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)

t1.start()
t2.start()

t1.join()
t2.join()

print(counter)  # wynik często ≠ 200000

```

Lock (mutex – mutual exclusion) to mechanizm, który pozwala tylko jednemu wątkowi na dostęp do danego fragmentu kodu w tym samym czasie. Czyli:

„Jeśli jeden wątek wykonuje sekcję chronioną przez lock, to drugi wątek musi czekać, aż pierwszy zwolni lock”

```

import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter
    for _ in range(100000):
        with lock: # blokada
            counter += 1

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)

t1.start()
t2.start()

t1.join()
t2.join()

```

```
print(counter) # zawsze 200000
```

```
# Bez locka wystąpiłby race condition (wyścig wątków)
```

Wieloprocusowość omija ograniczenia GIL poprzez tworzenie oddzielnych procesów, każdy z własnym interpreterem Pythona i przestrzenią pamięci.

Podstawy multiprocessing

Przykład:

Tworzysz **dwa osobne procesy**, które:

- wykonują ciężkie obliczenia (CPU)
- działają **równolegle** (nie jak wątki)

```
from multiprocessing import Process
import os
import time

def cpu_task(name, duration):
    pid = os.getpid() # ID procesu
    print(f"Proces {name} (PID: {pid}) startuje")

    # Intensywne obliczenia CPU
    result = 0
    for i in range(10000000):
        result += i ** 2

    time.sleep(duration)
    print(f"Proces {name} kończy")

if __name__ == "__main__":
    p1 = Process(target=cpu_task, args=("A", 2))
    p2 = Process(target=cpu_task, args=("B", 1))

    p1.start()
    p2.start()

    p1.join()
    p2.join()
```


Tworzenie procesów

1. Klasa Process

```
from multiprocessing import Process

def worker(name):
    print(f"Pracownik: {name}")

if __name__ == "__main__":
    # Pojedynczy proces
    p = Process(target=worker, args=("Jan",))
    p.start()
    p.join()

    # Wiele procesów
    processes = []
    for i in range(4):
        p = Process(target=worker, args=(f"P{i}",))
        processes.append(p)
        p.start()

    for p in processes:
        p.join()
```

2. Dziedziczenie po Process

```
from multiprocessing import Process

class CustomProcess(Process):
    def __init__(self, name):
        super().__init__()
        self.name = name

    def run(self):
        print(f"Proces {self.name} działa")
        # Tu kod procesu

if __name__ == "__main__":
    p = CustomProcess("Worker-1")
    p.start()
    p.join()
```

Komunikacja między procesami

Queue (kolejka)

```
from multiprocessing import Process, Queue
import time

def producer(queue):
    for i in range(5):
        queue.put(f"Dane-{i}")
        print(f"Wyprodukowano: Dane-{i}")
        time.sleep(0.5)
    queue.put(None) # Sygnał końca

def consumer(queue):
    while True:
        item = queue.get()
        if item is None:
            break
        print(f"Skonsumowano: {item}")
        time.sleep(1)

if __name__ == "__main__":
    q = Queue()

    p1 = Process(target=producer, args=(q,))
    p2 = Process(target=consumer, args=(q,))

    p1.start()
    p2.start()

    p1.join()
    p2.join()
```

Pipe (potok)

```
from multiprocessing import Process, Pipe

def sender(conn):
    conn.send("Wiadomość od nadawcy")
    conn.send(["lista", "danych", 123])
    conn.close()

def receiver(conn):
    while True:
        try:
            msg = conn.recv()
            print(f"Otrzymano: {msg}")
```

```

        except EOFError:
            break

if __name__ == "__main__":
    parent_conn, child_conn = Pipe()

    p = Process(target=sender, args=(child_conn,))
    p.start()

    receiver(parent_conn)
    p.join()

```

Współdzielenie danych

Value i Array (shared memory)

```

from multiprocessing import Process, Value, Array

def worker(val, arr):
    val.value += 10
    for i in range(len(arr)):
        arr[i] *= 2

if __name__ == "__main__":
    # Współdzielone dane
    shared_val = Value('i', 5)      # 'i' = integer
    shared_arr = Array('i', [1, 2, 3, 4]) # tablica intów

    p = Process(target=worker, args=(shared_val, shared_arr))
    p.start()
    p.join()

    print(f"Wartość: {shared_val.value}") # 15
    print(f"Tablica: {shared_arr[:]})"    # [2, 4, 6, 8]

```

Typy dla Value/Array:

- 'i' - signed int
- 'd' - double (float)
- 'f' - float
- 'c' - char
- 'b' - signed char

To fundamentalna różnica w Pythonie:

- **wielowątkowość** (*multithreading*) = wiele wątków w jednym procesie,
- **wieloprocusowość** (*multiprocessing*) = wiele niezależnych procesów.

Lekcja

Temat: Biblioteki zewnętrzne: Instalacja pip, numpy (tablice), pandas (dane tabularne).

pip to domyślny menedżer pakietów Pythona. Służy do instalowania bibliotek zewnętrznych (tzw. *third-party packages*).

1. Sprawdzenie wersji Python w terminalu / CMD

Najczęściej używane polecenie:

```
python -version
```

lub krócej:

```
python -V
```

2. Sprawdź czy pip istnieje

W terminalu wpisz:

```
python -m pip --version
```

Jeśli zobaczysz wersję, np.:

```
pip 25.0 from ...
```

to pip już jest.

3. Pobierz instalator pip

Pobierz plik:

[get-pip.py](https://bootstrap.pypa.io/get-pip.py)

Zapisz go np. w folderze:

Downloads

4. Otwórz terminal w folderze z plikiem

Przejdź do katalogu, np.:

Windows:

```
cd Downloads
```

5. Uruchom instalację pip

Wpisz:

```
python get-pip.py
```

Python pobierze i zainstaluje pip.

6. Sprawdź czy instalacja się udała

Po zakończeniu:

```
pip -V
```

lub:

```
python -m pip -V
```

Przykładowy wynik:

```
pip 25.0 from ...
```

Podstawowe komendy:

Instalacja jednej biblioteki

```
pip install numpy
```

```
pip install pandas
```

Instalacja wielu bibliotek naraz

```
pip install numpy pandas matplotlib seaborn
```

Instalacja konkretnej wersji

```
pip install numpy==1.26.4
```

```
pip install pandas>=2.0.0
```

Aktualizacja biblioteki

```
pip install --upgrade numpy
```

Odinstalowanie

```
pip uninstall numpy
```

NumPy – tablice (arrays)

NumPy to podstawowa biblioteka do obliczeń naukowych. Najważniejszym obiektem jest **ndarray** – wielowymiarowa tablica o stałym typie danych.

Instalacja

pip install numpy

lub

python -m pip install numpy

```
import numpy as np

# Tworzenie tablic
a = np.array([1, 2, 3, 4, 5])
b = np.array([[1, 2, 3], [4, 5, 6]])      # macierz 2x3
c = np.zeros((3, 4))                     # tablica zer
print("a =", a)
print("b =", b)
print(c)
print(f'np.zeros((3, 4)) = \n{np.zeros((3, 4))}')

d = np.ones((2, 2))                      # tablica jedynek
e = np.arange(0, 10, 2)                  # jak range, ale zwraca tablicę
f = np.linspace(0, 1, 5)                 # 5 równomiernie rozłożonych punktów

print("d = ", d)
print("e = ", e)
print(b.shape)      # wymiary → (2, 3) 2 wiersze i 3 kolumny → (2, 3)
print(b.ndim)       # liczba wymiarów → 2

x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

print(x + y)        # [5 7 9]
print(x * 10)       # [10 20 30]
print(x ** 2)       # [1 4 9]

# Operacje statystyczne
print(x.mean())
print(x.sum())
print(x.max())

A1 = np.array([[1, 2], [3, 4]])
B1 = np.array([[5, 6], [7, 8]])

print(A1 @ B1)      # mnożenie macierzowe (nowy operator @)
# lub np.dot(A1, B1)
```

```
...  
Mnożenie macierzowe:  
  
1. Element [0,0] → lewy górny róg  
Wiersz 0 z A: [1, 2]  
Kolumna 0 z B: [5, 7]  
     $1 \times 5 + 2 \times 7 = 5 + 14 = 19$   
  
2. Element [0,1] → prawy górny róg  
Wiersz 0 z A: [1, 2]  
Kolumna 1 z B: [6, 8]  
     $1 \times 6 + 2 \times 8 = 6 + 16 = 22$   
  
3. Element [1,0] → lewy dolny róg  
Wiersz 1 z A: [3, 4]  
Kolumna 0 z B: [5, 7]  
     $3 \times 5 + 4 \times 7 = 15 + 28 = 43$   
4. Element [1,1] → prawy dolny róg  
Wiersz 1 z A: [3, 4]  
Kolumna 1 z B: [6, 8]  
     $3 \times 6 + 4 \times 8 = 18 + 32 = 50$   
  
...
```

Pandas – dane tabelaryczne (DataFrame)

Pandas to biblioteka do pracy z danymi w formie tabel (jak Excel lub SQL). Główny obiekt to DataFrame i Series.

Instalacja

pip install pandas

lub

python -m pip install pandas

```
import pandas as pd  
  
# Tworzenie DataFrame  
data = {  
    'imie': ['Anna', 'Jan', 'Ola', 'Tomek'],  
    'wiek': [25, 30, 22, 35],  
}
```

```

    'miasto': ['Warszawa', 'Kraków', 'Gdańsk', 'Wrocław']
}

df = pd.DataFrame(data)

print(f'df = {df}')

print(df.head())                # pierwsze 5 wierszy
print()
print(f'info {df.info()}')      # informacje o typach i brakach
print()

# Wybieranie kolumn
print(df['imie'])
print(df[['imie', 'wiek']])

# Filtrowanie
print(df[df['wiek'] > 25])

# Dodawanie nowej kolumny
df['wiek_z_5_lat'] = df['wiek'] + 5
print(f"Wiek za 3 lata { df['wiek'] + 3 } ")
# Grupowanie
print(df.groupby('miasto')['wiek'].mean())

```

Lekcja

Temat: Dekoratory zaawansowane: Dekoratory z argumentami, functools.wraps.

Dekorator to funkcja, która przyjmuje inną funkcję jako argument, dodaje do niej jakąś dodatkową funkcjonalność (przed, po lub wokół jej wykonania), a następnie zwraca zmodyfikowaną wersję tej funkcji. Dekoratory pozwalają **rozszerzać zachowanie funkcji** bez zmieniania jej kodu źródłowego – jest to realizacja zasady **Open/Closed**.

```

def dekorator(func):
    def wrapper():
        print("Przed wywołaniem funkcji")
        func()
        print("Po wywołaniu funkcji")

```



```

    return wrapper

def przykład():
    print("Jestem oryginalną funkcją")

przykład = dekorator(przykład)
przykład()

```

Wygodniejsza składnia (używana w praktyce):

```

def dekorator(func):
    def wrapper():
        print("Przed wywołaniem funkcji")
        func()
        print("Po wywołaniu funkcji")
    return wrapper

@dekorator                                # to jest równoważne z: przykład = dekorator(przykład)
def przykład():
    print("Jestem oryginalną funkcją")

przykład()

```

1. Dekoratory z argumentami (Decorator factories)

Zwykły dekorator nie przyjmuje argumentów. Gdy chcemy przekazać parametry do dekoratora, musimy stworzyć **fabrykę dekoratorów** – czyli funkcję, która zwraca dekorator.

```

from functools import wraps

# Twój dekorator
def log_execution(show_args=True, show_result=True):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            print(f"> Wywołuję {func.__name__}()")

            if show_args:
                print(f"    Argumenty: {args} {kwargs}")

            result = func(*args, **kwargs)

```

```

        if show_result:
            print(f"    Wynik: {result}")

        print(f"← Zakończono {func.__name__}()\n")
        return result
    return wrapper
return decorator

# Funkcja z dekoratorem
@log_execution(show_args=True, show_result=False)
def oblicz_cene(netto, vat=0.23):
    return round(netto * (1 + vat), 2)

cena = oblicz_cene(100)
print(f"Zwrócona wartość: {cena}")

cena2 = oblicz_cene(50, vat=0.08)
cena3 = oblicz_cene(200, 0.05)

```

`round(liczba, liczba_miejsc)`

- `liczba` — wartość do zaokrąglenia
- `liczba_miejsc` — opcjonalna liczba miejsc po przecinku

Przykład 1: Dekorator z argumentami

```

from functools import wraps

def info(tekst):
    def decorator(func):
        @wraps(func)
        def newFunWrapper(*args, **kwargs):
            print(f"[{tekst}]")
            print(f"Uruchamiam funkcję: {func.__name__}")
            wynik = func(*args, **kwargs)
            print(f"Zakończono {func.__name__}\n")
            return wynik
        return newFunWrapper
    return decorator

@info("WAŻNE ZADANIE")
def dodaj(a, b):
    return a + b

print(dodaj(5, 7))

```

@wraps(func) to jeden z **najważniejszych** elementów przy pisaniu dekoratorów.

@wraps(func) - Gdy stworzysz dekorator, funkcja wrapper zastępuje oryginalną funkcję. Bez @wraps Python "zapomina", że to tak naprawdę Twoja oryginalna funkcja i przypisuje jej dane wrappera. @wraps(func) kopiuje ważne informacje z oryginalnej funkcji do wrappera:

- __name__ (nazwa funkcji)
- __doc__ (docstring / dokumentacja)
- __annotations__ (podpowiedzi typów)
- __module__
- sygnaturę funkcji (help() i inspect.signature() działają poprawnie)

@wraps(func) - sprawia, że **funkcja po dekoracji wygląda jak oryginalna funkcja**

✗ Bez @wraps — problem

```
from functools import wraps

def dekorator(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@dekorator
def dodaj(a, b):
    "Dodaje liczby"
    return a + b

print(dodaj.__name__)
print(dodaj.__doc__)
```

Wynik:

wrapper

None

Python „widzi” tylko funkcję wrapper, a nie dodaj

☑ Z @wraps(func) — poprawnie

```
from functools import wraps

def dekorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
```

```
    return wrapper

@dekorator
def dodaj(a, b):
    "Dodaje liczby"
    return a + b

print(dodaj.__name__)
print(dodaj.__doc__)
```

Wynik:

```
dodaj
Dodaje liczby
```

Lekcja

Temat: Walrus operator (:=), type hints (typing module).

Walrus operator :=

Pozwala przypisać wartość do zmiennej **i jednocześnie użyć jej w wyrażeniu** (np. w warunku if lub pętli while). Skracza kod i eliminuje potrzebę osobnej linii z przypisaniem.

Przykłady zastosowań:

a) W pętli while – uniknięcie dwukrotnego wołania funkcji

```
# Bez := (tradycyjnie)
data = input("Wpisz coś: ")
while data != "quit":
    print(f"Wpisałeś: {data}")
    data = input("Wpisz coś: ")

# Z :=
while (data := input("Wpisz coś: ")) != "quit":
    print(f"Wpisałeś: {data}")
```

b) W if – sprawdzenie i użycie wyniku funkcji

```
import re

# Bez :=
match = re.search(r"\d+", "Numer 42")
r'''
wyrażenie regularne (regex) \d+ oznacza znajdź jedną lub więcej cyfr
'''
if match:
    print(f"Znaleziono: {match.group()}") # group() zwraca dopasowany tekst, czyli "42"

# Z :=
if (match := re.search(r"\d+", "Numer 42")):
    print(f"Znaleziono: {match.group()}")
```

c) List comprehension – filtrowanie z zachowaniem wyniku

```
# Bez := - trzeba dwa razy liczyć
wyniki = [x for x in [len(y) for y in lista] if x > 2]

# Z :=
wyniki = [dl for y in lista if (dl := len(y)) > 2]
```

Type hints, moduł typing

1. Podstawowe type hints (bez typing)

```
def add(a: int, b: int) -> int:
    return a + b

name: str = "Jan"
age: int = 30
is_active: bool = True
```

a: int – argument typu int
-> int – zwracany typ

◇ 2. Listy, słowniki, tuple (moduł typing)

W starszych wersjach Pythona używa się typing.

```
from typing import List, Dict, Tuple
```

```
numbers: List[int] = [1, 2, 3]
users: Dict[str, int] = {"Jan": 30, "Anna": 25}
point: Tuple[int, int] = (10, 20)
```

◇ 3. Python 3.9+ (nowoczesny zapis bez typing)

Od Pythona 3.9 możesz pisać prościej:

```
numbers: list[int] = [1, 2, 3]
users: dict[str, int] = {"Jan": 30}
point: tuple[int, int] = (10, 20)
```

◇ 4. Optional (wartość może być None)

```
from typing import Optional
def find_user(user_id: int) -> Optional[str]:
    if user_id == 1:
        return "Jan"
    return None
```

Nowy zapis (Python 3.10+):

```
def find_user(user_id: int) -> str | None:
    if user_id == 1:
        return "Jan"
    return None
```

◇ 5. Union (kilka możliwych typów)

```
from typing import Union
def process(value: Union[int, str]):
    print(value)
```

Nowocześnie:

```
def process(value: int | str):
    print(value)
```

◇ 6. Any (dowolny typ)

```
from typing import Any
def debug(value: Any):
```

```
print(value)
```

👉 brak kontroli typu

◇ 7. Callable (funkcje jako argumenty)

```
from typing import Callable

def calculate(a: int, b: int, func: Callable[[int, int], int]) -> int:
    return func(a, b)

def multiply(x: int, y: int) -> int:
    return x * y

result = calculate(2, 3, multiply)
```

◇ 8. TypedDict (słownik z określoną strukturą)

```
from typing import TypedDict

class User(TypedDict):
    name: str
    age: int

user: User = {
    "name": "Jan",
    "age": 30
}
```

◇ 9. Literal (ograniczone wartości)

```
from typing import Literal

def set_mode(mode: Literal["dev", "prod"]):
    print(mode)
```

👉 tylko "dev" albo "prod"

◇ 10. Type Alias (alias typów)

```
from typing import List

Vector = List[int]

def sum_vector(v: Vector) -> int:
    return sum(v)
```

◇ 11. Praktyczny przykład (mini system)

```
from typing import List, Optional

class Product:
    def __init__(self, name: str, price: float):
        self.name = name
        self.price = price

def find_product(products: List[Product], name: str) -> Optional[Product]:
    for p in products:
        if p.name == name:
            return p
    return None
```

Lekcja

Temat: Introspekcja. Metaklasy

Introspekcja (introspection) w Pythonie to możliwość badania obiektu w czasie działania programu (runtime) – sprawdzanie jego typu, atrybutów, metod itp.

1. type()

type(obiekt) zwraca typ (klasę) obiektu.


```
class PrzykładowaKlasa:
    pass

obj = PrzykładowaKlasa()

print(type(obj))           # <class '__main__.PrzykładowaKlasa'>
print(type(PrzykładowaKlasa)) # <class 'type'>
print(type(5))             # <class 'int'>
print(type(int))           # <class 'type'>
```

Klasy w Pythonie są obiektami, a ich typem (klasą) jest **metaklasa** type.

int też jest klasą, czyli **obiekt klasy** (dokładniej: instancja metaklasy type).

2. dir()

dir(obiekt) zwraca listę nazw atrybutów i metod obiektu (w tym odziedziczonych).

```
class MojaKlasa:
    def moja_metoda(self):
        pass

obj = MojaKlasa()
print(dir(obj))
'''
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
 '__firstlineno__', '__format__', '__ge__', '__getattribute__', '__getstate__', '__gt__',
 '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__',
 '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
 '__static_attributes__', '__str__', '__subclasshook__', '__weakref__', 'moja_metoda']
<class '__main__.MojaKlasa'>
'''
```

Te nazwy typu `__delattr__`, `__gt__`, `__lt__` itd. to tzw. **metody specjalne** (ang. *magic methods* albo *dunder methods* — od *double underscore*).

3. getattr()

getattr(obiekt, 'nazwa_atrybutu', domyślna_wartość) pobiera atrybut po nazwie (string).

```
class Osoba:
    imie = "Jan"

p = Osoba()
```

```
print(getattr(p, 'imie'))          # Jan
# print(getattr(p, 'nazwisko'))    # AttributeError: 'Osoba' object has no attribute
# 'nazwisko'
print(getattr(p, 'nazwisko', 'Brak')) # Brak (nie rzuca AttributeError)
```

Istnieją też setattr(), setattr() i delattr().

Przykład praktyczny introspekcji:

```
class Osoba:
    imie = "Jan"

p = Osoba()
def wyswietl_atrybuty(obj):
    print(f"Typ: {type(obj)}")
    print("Atrybuty:")
    for attr in dir(obj):
        if not attr.startswith('__'):
            value = getattr(obj, attr, None)
            print(f"    {attr} = {value}")

wyswietl_atrybuty(p)
```

```
PS C:\PYTHON\lekcje> python lek.py
```

```
Typ: <class '__main__.Osoba'>
```

```
Atrybuty:
```

```
    imie = Jan
```

```
PS C:\PYTHON\lekcje>
```

Metaklasy (Metaclasses)

Metaklasa to "klasa klas". Kontroluje, jak powstaje klasa. Kiedy piszesz:

```
class MojaKlasa:
    pass
```

Python robi w tle coś takiego:

```
MojaKlasa = type('MojaKlasa', (), {})
```

type to domyślna metaklasa.

Definiowanie własnej metaklasy

Najczęściej nadpisujemy `__new__` metaklasy (tworzy klasę).

```
class Meta(type): # metaklasa musi dziedziczyć po type
    def __new__(cls, name, bases, dct):
        # dct = słownik atrybutów klasy
        print(f"Tworzę klasę {name}")

        # Modyfikujemy klasę przed jej stworzeniem
        dct['dodany_atrybut'] = "Dodane przez metaklasę!"
        dct['wersja'] = 2

        # Tworzymy klasę
        return super().__new__(cls, name, bases, dct)

class MojaKlasa(metaclass=Meta):
    pole = "normalne"

print(MojaKlasa.dodany_atrybut) # Dodane przez metaklasę!
print(MojaKlasa.wersja)        # 2
```

MojaKlasa jest obiektem metaklasy Meta

W Python:

- zwykła klasa tworzy obiekty
- metaklasa tworzy klasy

```
class A:
    pass

obj = A()
```

- $A \rightarrow$ klasa
- $obj \rightarrow$ obiekt klasy A

Natomiast:

```
class Meta(type):  
    pass
```

Meta jest klasą tworzącą klasy.